Architecture 1001: x86-64 Assembly IfExample2.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

[OO_so_below_X]

IfExample2.c

```c
int main() {
  unsigned int a = -1, b = 2;
  if (a == b) {
    return 1;
  }
  if (a > b) {
    return 2;
  }
  if (a < b) {
    return 3;
  }
  return 0xdefea7ed;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],0FFFFFFFFh
000000014000100C  mov  dword ptr [rsp],2
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  cmp  dword ptr [rsp+4],eax
000000014000101A  jne  0000000140001023
000000014000101C  mov  eax,1
0000000140001021  jmp  0000000140001048
0000000140001023  mov  eax,dword ptr [rsp]
0000000140001026  cmp  dword ptr [rsp+4],eax
000000014000102A  jbe  0000000140001033
000000014000102C  mov  eax,2
0000000140001031  jmp  0000000140001048
0000000140001033  mov  eax,dword ptr [rsp]
0000000140001036  cmp  dword ptr [rsp+4],eax
000000014000103A  jae  0000000140001043
000000014000103C  mov  eax,3
0000000140001041  jmp  0000000140001048
0000000140001043  mov  eax,0DEFEA7EDh
0000000140001048  add  rsp,18h
000000014000104C  ret
```

[OO_so_below_X]

[OO_so_below_X]
[X_evoqe_se_OO]

IfExample2.c

```
int main() {
  unsigned int a = -1, b = 2;
  if (a == b) {
    return 1;
  }
  if (a > b) {
    return 2;
  }
  if (a < b) {
    return 3;
  }
  return 0xdefea7ed;
}
```

Jcc

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],0FFFFFFFFh
000000014000100C  mov  dword ptr [rsp],2
0000000140001013  mov  eax,dword ptr [rsp]
0000000140001016  cmp  dword ptr [rsp+4],eax
000000014000101A  jne  0000000140001023
000000014000101C  mov  eax,1
0000000140001021  jmp  0000000140001048
0000000140001023  mov  eax,dword ptr [rsp]
0000000140001026  cmp  dword ptr [rsp+4],eax
000000014000102A  jbe  0000000140001033
000000014000102C  mov  eax,2
0000000140001031  jmp  0000000140001048
0000000140001033  mov  eax,dword ptr [rsp]
0000000140001036  cmp  dword ptr [rsp+4],eax
000000014000103A  jae  0000000140001043
000000014000103C  mov  eax,3
0000000140001041  jmp  0000000140001048
0000000140001043  mov  eax,0DEFEA7EDh
0000000140001048  add  rsp,18h
000000014000104C  ret
```

[OO_so_below_X]
[X_evoqe_se_OO]

Some Notable Jcc Instructions 3

- Mnemonic translations
- **B** = below, unsigned notion
- **A** = above, unsigned notion
- **N** = Not (like “Not less than:” JNL)
- **G** = greater than, signed notion
- **L** = less than, signed notion
- **E** = Equal (same a Z, zero flag set)

Inference…

Inference…

- There are different conditions for unsigned (above) vs. signed (greater than)…

Inference…

- There are different conditions for unsigned (above) vs. signed (greater than)…
- Which leads to different assembly instructions for unsigned (JA) vs. signed (JG) comparisons…

Inference…

- There are different conditions for unsigned (above) vs. signed (greater than)…
- Which leads to different assembly instructions for unsigned (JA) vs. signed (JG) comparisons…
- Which implies the compiler emits different code depending on whether the programmer declared variables as unsigned vs. signed…

Inference…

- There are different conditions for unsigned (above) vs. signed (greater than)…
- Which leads to different assembly instructions for unsigned (JA) vs. signed (JG) comparisons…
- Which implies the compiler emits different code depending on whether the programmer declared variables as unsigned vs. signed…
- Which a reverse engineer / decompiler can use to infer whether variables are likely unsigned or signed

Less Obvious

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**
- Basically the hardware doesn't know or care about whether the humans are currently interpreting the bits as signed or unsigned. That's the compiler's problem to sort out.

Less Obvious

- It turns out that for instructions that set status flags (e.g. arithmetic operations), the *hardware* just does the operation and sets flags as if the operands were **both unsigned and signed**
- Basically the hardware doesn't know or care about whether the humans are currently interpreting the bits as signed or unsigned. That's the compiler's problem to sort out.
- The compiler must emit instructions which treat the bits as signed or unsigned based on what's specified in the high level language



STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
CHECK YOUR UNDERSTANDING



STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
CHECK YOUR UNDERSTANDING